```
function [param,acheck_array,model_info,ens_coh] =
ps_periodogram_estimation(h2ph,Btemp,dphase,model,std_param,step1_orig,step2_ori
g)

% Function for the estimation of the DEM (error) and linear
% deformation using the ambiguity function.
%
% Input:  - h2ph               height to phase factors
%         - Btemp              temporal baselines [year]
%         - dphase             differential phases
%         - model              vector with model indices to be
%                              evaluated (see help ps_model_definitions
%                              for more information)
%         - std_param          standard deviations of parameters
%         - step1_orig         step size for topo
%         - step2_orig         step size for linear deformation
%
% Output: - param              estimated parameters
%         - acheck_array       estimated ambiguities
%         - model_info         information about the used model
%         - ens_coh            ensemble coherence
%
% ------------------------------------------------------------------------
% File............: ps_periodogram_estimation.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Author..........: Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
%


% ------------------------------------------------------------------------
% Initialize
% ------------------------------------------------------------------------

global althyp_index Npar_max

[Npoints,Nifgs] = size(dphase);
model = 1; %only linear model
Nmodels = length(model);
if model(1)~=1
    error('Only a linear model is supported for the periodogram technique');
end
model_index = 1;
althyp_index = 2; % for proper flow of models

model_info = repmat(NaN,Npoints,2);
ens_coh = repmat(NaN,Npoints,1);
param = repmat(NaN,Npoints,Npar_max);
acheck_array = repmat(NaN,Npoints,Nifgs);
mean_h2ph = mean(h2ph,1)';

% ------------------------------------------------------------------------
% Main loop to test alternative hypothesis
% ------------------------------------------------------------------------
```

```
points_todo= 1:Npoints;

while ~isempty(points_todo) && (model_index<=Nmodels)

  fprintf(1,'Evaluating model %g of %g\n',model_index,Nmodels);
  fprintf(1,'Model: %g, alternative hypothesis:
%g\n',model(model_index),althyp_index);
  fprintf(1,'Points remaining: %g of %g\n\n',length(points_todo),Npoints);

  % ------------------------------------------------------------------------
  % Construct search space
  % ------------------------------------------------------------------------

  [Npar,par_index] = ps_model_definitions('Npar',model(model_index));
  [B1Qy2,par_index] =
ps_model_definitions('model',model(model_index),Nifgs,mean_h2ph,Btemp,
[],std_param);
  B1 = B1Qy2(1:Nifgs,:);
  N = B1'*B1;
  R = chol(N);
  rhs = R\(R'\B1');
  std_par = sqrt(diag(B1Qy2(Nifgs+1:end,:)));

  param1 = 0;
  param2 = 0;
  Nsearch1_orig = round(2*std_par(1)/step1_orig);
  Nsearch2_orig = round(2*std_par(2)/step2_orig);
  search_space1 = B1(:,1)*(param1-
Nsearch1_orig*step1_orig:step1_orig:param1+Nsearch1_orig*step1_orig);
  search_space2 = B1(:,2)*(param2-
Nsearch2_orig*step2_orig:step2_orig:param2+Nsearch2_orig*step2_orig);
  Ssearch1_orig = size(search_space1,2);
  Ssearch2_orig = size(search_space2,2);
  search_space_orig = exp(-i*(kron(search_space1,ones(1,Ssearch2_orig))+...
                              kron(ones(1,Ssearch1_orig),search_space2)));



  % ------------------------------------------------------------------------
  % Main loop
  % ------------------------------------------------------------------------

  for v = points_todo

      %dummy = [v length(points_todo)];
      %if (rem(v,1000)==0)
      %    fprintf(1,'Processing point %5.0f of %5.0f\n',dummy);
      %end

      step1 = step1_orig;
      step2 = step2_orig;
      Nsearch1 = Nsearch1_orig;
      Nsearch2 = Nsearch2_orig;
      Ssearch1 = Ssearch1_orig;
      Ssearch2 = Ssearch2_orig;
      param1 = 0;
      param2 = 0;
      search_space = search_space_orig;

      count = 0;


      % ------------------------------------------------------------------------
```

```
      % Search loop
      % --------------------------------------------------------------------

      while (step1 > 1e-4 ) && (step2 > 1e-7) && (count < 10)

          count = count + 1;

          [best,index] = max( (1/Nifgs)*(exp(i*dphase(v,:))*search_space) );
          [cn2,cn1] = ind2sub([Ssearch2 Ssearch1],index);


          %
----------------------------------------------------------------------
          % Zoom in
          %
----------------------------------------------------------------------

          param1 = param1 + (cn1-(Nsearch1+1))*step1;
          param2 = param2 + (cn2-(Nsearch2+1))*step2;
          step1 = step1/10;
          step2 = step2/10;
          Nsearch1 = 11;
          Nsearch2 = 11;

          search_space1 = B1(:,1)*(param1-
Nsearch1*step1:step1:param1+Nsearch1*step1);
          search_space2 = B1(:,2)*(param2-
Nsearch2*step2:step2:param2+Nsearch2*step2);
          Ssearch1 = size(search_space1,2);
          Ssearch2 = size(search_space2,2);
          search_space = exp(-i*(kron(search_space1,ones(1,Ssearch2))+...
                                 kron(ones(1,Ssearch1),search_space2)));

      end %while

      ens_coh(v) = abs(best);

      % --------------------------------------------------------------------
      % Correct estimated dH's for approximate h2ph
      % --------------------------------------------------------------------

      factors = h2ph(v,:)./mean_h2ph';
      param(v,par_index(1)) = param1/median(factors);
      param(v,par_index(2)) = param2;

      %%% calculate the ambiguities (required for unwrapping);
      model_est = (B1*param(v,par_index)')' + phase(best);
      dphase_new = dphase(v,:)-model_est;

      %%% wrap again, because the corrections were unwrapped
      dphase_new = mod(dphase_new+pi,2*pi)-pi;

      %%% calculate acheck, dphase + 2pi*a = H+D1+noise
      acheck_array(v,:) = round((model_est+dphase_new-dphase(v,:))/(2*pi));

      dphase_unw = 2*pi*acheck_array(v,:)+dphase(v,:);
      param(v,par_index) = (rhs*dphase_unw')';

  end %for

  model_info(:,1) = ones(Npoints,1);
  model_index = model_index+1;
end
```